HDOC DAY-71

Write a C program to dynamically allocate two matrices and perform addition, subtraction, and multiplication using pointer notation. Implement each operation as a separate function and release all allocated memory properly.

Algorithm: Dynamic Matrix Operations Using Pointers

1. Start.
2. Declare variables for the number of rows and columns of Matrix A and Matrix B.
3. Read r1, c1, r2, and c2 from the user.
4. Dynamically allocate memory for Matrix A and Matrix B using malloc().
5. Check whether memory allocation is successful.
6. Read the elements of Matrix A and Matrix B using pointer notation *(*(mat + i) + j).
7. For **addition:**
    - Check whether r1 == r2 and c1 == c2.
    - If true, dynamically allocate memory for the result matrix.
    - Add corresponding elements:
      C[i][j] = A[i][j] + B[i][j].
    - Display the result and free the result matrix.
    - Otherwise, display that addition is not possible.
8. For **subtraction**:
    - Check whether r1 == r2 and c1 == c2.
    - If true, dynamically allocate memory for the result matrix.
    - Subtract corresponding elements:
      C[i][j] = A[i][j] - B[i][j].
    - Display the result and free the result matrix.
    - Otherwise, display that subtraction is not possible.
9. For **multiplication**:
    - Check whether c1 == r2.
    - If true, dynamically allocate a result matrix of size r1 × c2.
    - Initialize every result element to 0.

- Calculate each element using:
  C[i][j] = C[i][j] + A[i][k] × B[k][j].
- Display the multiplication result and free the result matrix.
- Otherwise, display that multiplication is not possible.

10. Free the dynamically allocated memory of Matrix A and Matrix B using free().
11. Stop.

• C Program:

```
#include <stdio.h>
#include <stdlib.h>

// Function to allocate memory for a matrix
int **allocateMatrix(int rows, int cols)
{
    int **mat = (int **)malloc(rows * sizeof(int *));

    if (mat == NULL)
    {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    for (int i = 0; i < rows; i++)
    {
        *(mat + i) = (int *)malloc(cols * sizeof(int));

        if (*(mat + i) == NULL)
        {
            printf("Memory allocation failed!\n");
            exit(1);
        }
    }

    return mat;
}

// Function to input matrix elements
```

```
void inputMatrix(int **mat, int rows, int cols)
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            scanf("%d", (*(mat + i) + j));
        }
    }
}

// Function to display a matrix
void displayMatrix(int **mat, int rows, int cols)
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            printf("%d\t", *(*(mat + i) + j));
        }
        printf("\n");
    }
}

// Function for matrix addition
int **addMatrix(int **A, int **B, int rows, int cols)
{
    int **C = allocateMatrix(rows, cols);

    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            *(*(C + i) + j) =
                *(*(A + i) + j) + *(*(B + i) + j);
        }
    }

    return C;
```

```
}

// Function for matrix subtraction
int **subtractMatrix(int **A, int **B, int rows, int cols)
{
   int **C = allocateMatrix(rows, cols);

   for (int i = 0; i < rows; i++)
   {
      for (int j = 0; j < cols; j++)
      {
         *(*(C + i) + j) =
            *(*(A + i) + j) - *(*(B + i) + j);
      }
   }

   return C;
}

// Function for matrix multiplication
int **multiplyMatrix(int **A, int **B,
                     int r1, int c1, int c2)
{
   int **C = allocateMatrix(r1, c2);

   for (int i = 0; i < r1; i++)
   {
      for (int j = 0; j < c2; j++)
      {
         *(*(C + i) + j) = 0;

         for (int k = 0; k < c1; k++)
         {
            *(*(C + i) + j) +=
               *(*(A + i) + k) *
               *(*(B + k) + j);
         }
      }
   }
```

```
    return C;
}

// Function to free dynamically allocated memory
void freeMatrix(int **mat, int rows)
{
    for (int i = 0; i < rows; i++)
    {
        free(*(mat + i));
    }

    free(mat);
}

int main()
{
    int r1, c1, r2, c2;
    int **A, **B, **result;

    printf("Enter rows and columns of Matrix A: ");
    scanf("%d %d", &r1, &c1);

    printf("Enter rows and columns of Matrix B: ");
    scanf("%d %d", &r2, &c2);

    A = allocateMatrix(r1, c1);
    B = allocateMatrix(r2, c2);

    printf("\nEnter elements of Matrix A:\n");
    inputMatrix(A, r1, c1);

    printf("\nEnter elements of Matrix B:\n");
    inputMatrix(B, r2, c2);

    // Addition
    if (r1 == r2 && c1 == c2)
    {
        result = addMatrix(A, B, r1, c1);
```

```
    printf("\nAddition of matrices:\n");
    displayMatrix(result, r1, c1);

    freeMatrix(result, r1);

    // Subtraction
    result = subtractMatrix(A, B, r1, c1);

    printf("\nSubtraction of matrices (A - B):\n");
    displayMatrix(result, r1, c1);

    freeMatrix(result, r1);
  }
  else
  {
    printf("\nAddition and subtraction are not possible.\n");
  }

  // Multiplication
  if (c1 == r2)
  {
    result = multiplyMatrix(A, B, r1, c1, c2);

    printf("\nMultiplication of matrices:\n");
    displayMatrix(result, r1, c2);

    freeMatrix(result, r1);
  }
  else
  {
    printf("\nMatrix multiplication is not possible.\n");
  }

  // Release matrices A and B
  freeMatrix(A, r1);
  freeMatrix(B, r2);

  return 0;
```

}

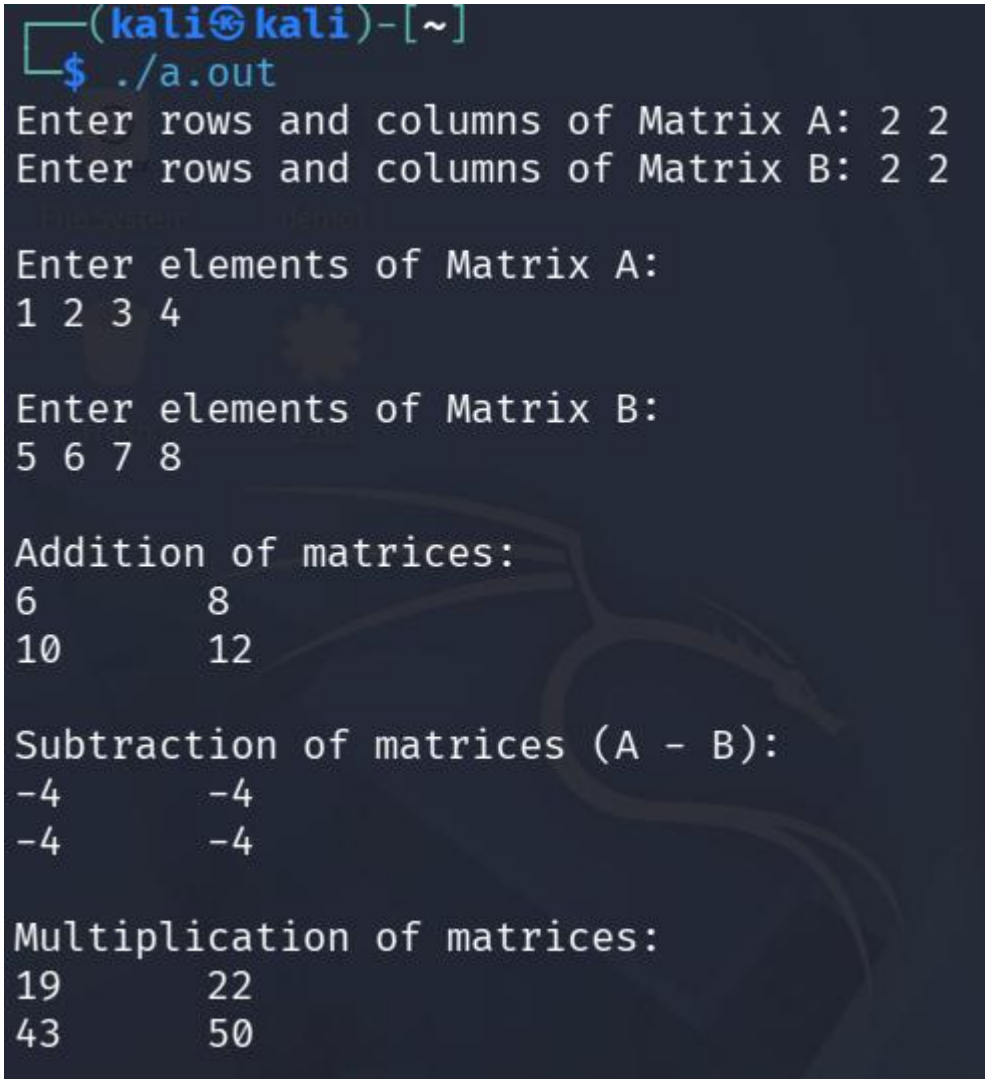
Test Case Table:

Test Case	Matrix A	Matrix B	Expected Addition	Expected Subtraction	Expected Multiplication	Remark
1.2*2 matrices	1 2 3 4	5 6 7 8	6 8 10 12	-4 -4 -4 -4	19 22 43 50	All three operations possible
2.1*1 matrices	5	3	8	2	15	Tests minimum matrix size
3.Rectangular matrices	1 2 3 4 5 6	7 8 9 10 11 12	Not Possible	Not Possible	58 64	Only multiplication possible
4.Different incompatible sizes	1 2 3 4	1 2 3	Not Possible	Not Possible	Not Possible	Tests invalid dimensions for all operations
5.Zero matrix	0 0	1 2 3 4	1 2 3 4	-1 -2 -3 -4	0 0 0 0	Tests operation involving zeroes
6.Negative elements	-1 2 3 4	5 -6 -7 8	4 -4 -4 4	-6 8 10 -12	-19 22 43 -50	Test negative values

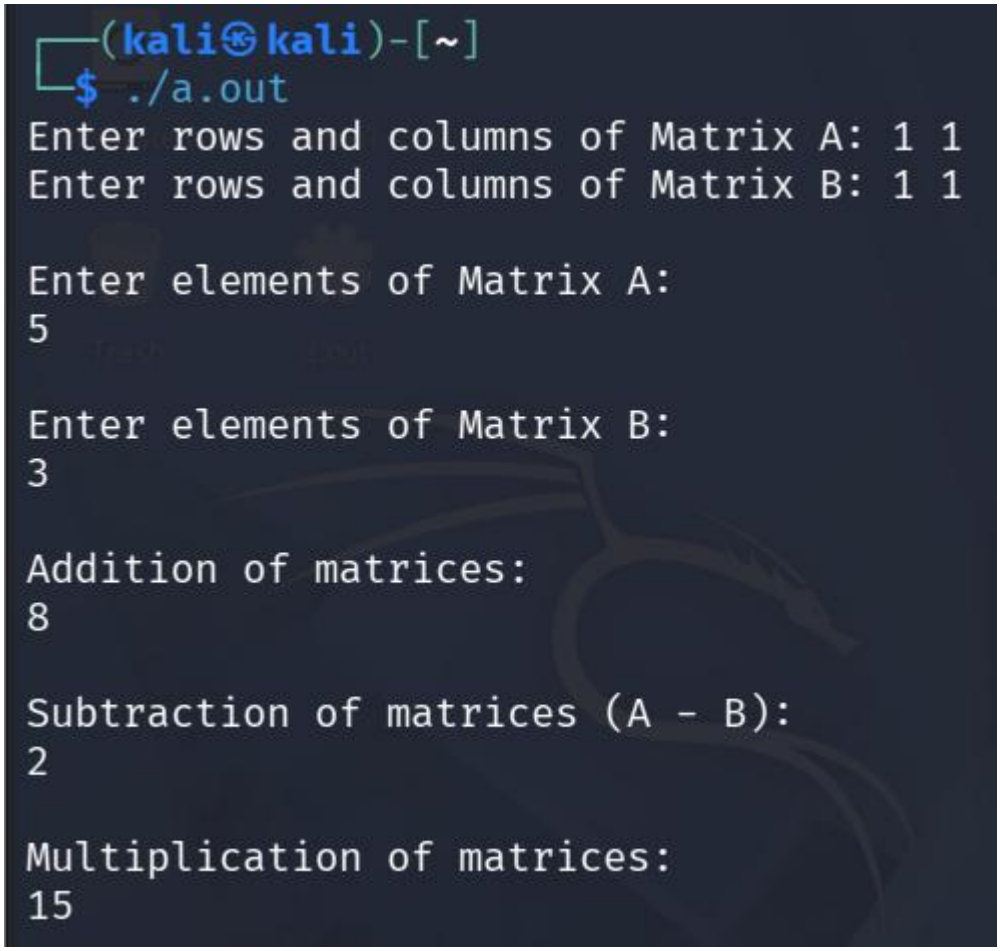
Compilation and Execution:

The following test cases has been compiled and executed successfully.

Test Case 1:


(kali㉿kali)-[~]
$ ./a.out
Enter rows and columns of Matrix A: 2 2
Enter rows and columns of Matrix B: 2 2
Enter elements of Matrix A:
1 2 3 4
Enter elements of Matrix B:
5 6 7 8
Addition of matrices:
6 8
10 12
Subtraction of matrices (A - B):
-4 -4
-4 -4
Multiplication of matrices:
19 22
43 50

Test Case 2:


(kali㉿kali)-[~]
$ ./a.out
Enter rows and columns of Matrix A: 1 1
Enter rows and columns of Matrix B: 1 1
Enter elements of Matrix A:
5
Enter elements of Matrix B:
3
Addition of matrices:
8
Subtraction of matrices (A - B):
2
Multiplication of matrices:
15

Test Case 3:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter rows and columns of Matrix A: 2 3
Enter rows and columns of Matrix B: 3 2

Enter elements of Matrix A:
1 2 3 4 5 6

Enter elements of Matrix B:
7 8 9 10 11 12

Addition and subtraction are not possible.

Multiplication of matrices:
58      64
139     154
```

Test Case 4:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter rows and columns of Matrix A: 2 2
Enter rows and columns of Matrix B: 1 1

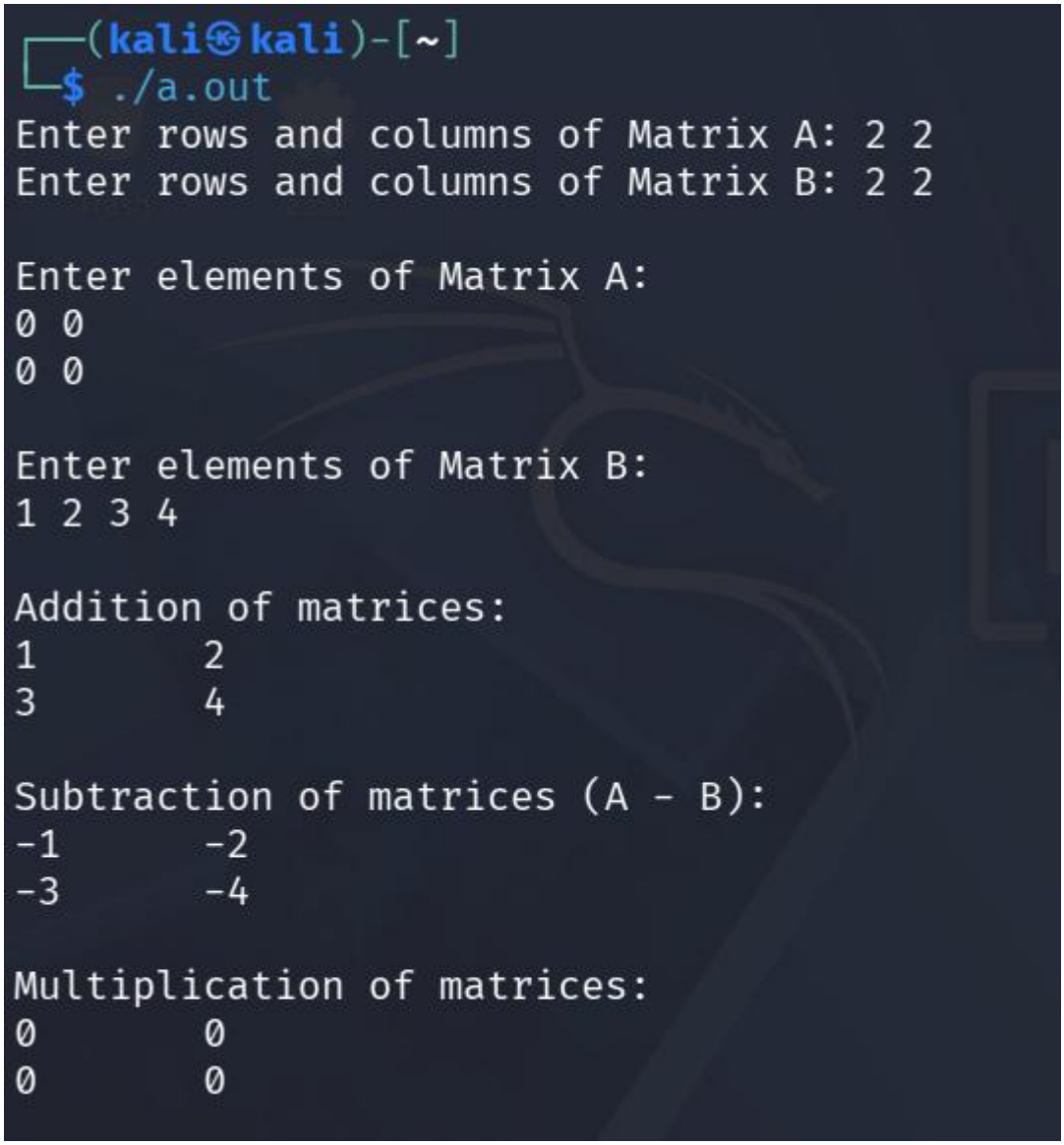
Enter elements of Matrix A:
1 2 3 4

Enter elements of Matrix B:
1 2 3

Addition and subtraction are not possible.

Matrix multiplication is not possible.
```

Test Case 5:



Test Case 6:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter rows and columns of Matrix A: 2 2
Enter rows and columns of Matrix B: 2 2

Enter elements of Matrix A:
-1 2 3 -4

Enter elements of Matrix B:
5 -6 -7 8

Addition of matrices:
4       -4
-4      4

Subtraction of matrices (A - B):
-6      8
10      -12

Multiplication of matrices:
-19     22
43      -50
```